

24.4 — Advanced Async Patterns & Modern JavaScript

Goal: Async JavaScript-এর advanced concepts বুঝে production-level async code লিখতে পারা — timeout, cancellation, retry, concurrency limiting, async iteration, microtasks, fire-and-forget, resource cleanup, `Promise.withResolvers()`, top-level `await` এবং async state management ।

1. Floating Promises

যখন কোনো Promise start করা হয় কিন্তু সেটাকে:

- `await` করা হয় না
- `.catch()` দেওয়া হয় না
- caller-এর কাছে `return` করা হয় না

তখন সেটাকে সাধারণত **floating promise** বলা হয় ।

```
saveUser(user);  
  
console.log("Done");
```

এখানে `saveUser()` যদি Promise return করে, তাহলে Promise-এর outcome handle করা হয়নি ।

1.1 কেন Floating Promise Problematic?

```
async function save() {  
  throw new Error("Save failed");  
}  
  
function main() {  
  save();  
  
  console.log("Finished");  
}  
  
main();
```

`save()` -এর rejection কোথাও handle করা হয়নি ।

ফলে environment অব্যাহী **unhandled rejection** হতে পারে।

2. Unhandled Promise Rejection

Promise reject হওয়ার পর যদি তার rejection handle করার জন্য কোনো rejection handler না থাকে, তাহলে সেটি unhandled rejection হতে পারে।

Common handling:

```
save().catch((error) => {  
  console.error(error);  
});
```

অথবা:

```
try {  
  await save();  
} catch (error) {  
  console.error(error);  
}
```

3. Floating Promise কখন Intentional হতে পারে?

কিছু কাজ intentionally background-এ চালানো হয়।

যেমন:

```
sendAnalyticsEvent();
```

কিন্তু intentional হলেও error handling strategy থাকা ভালো।

```
void sendAnalyticsEvent().catch((error) => {  
  console.error("Analytics failed:", error);  
});
```

এখানে `void` দিয়ে বোঝানো হচ্ছে:

এই Promise-এর result intentionally consume করছি না।

তবে project-এর linting rules অনুযায়ী pattern আলাদা হতে পারে।

4. Sequential vs Concurrent vs Parallel

এই তিনটি concept আলাদা।

Sequential

$A \rightarrow B \rightarrow C$

একটি শেষ হওয়ার পর অন্যটি শুরু।

```
await A();  
await B();  
await C();
```

Concurrent

A _____
B _____
C _____

একাধিক async operation overlappingভাবে progress করছে।

```
await Promise.all([  
  A(),  
  B(),  
  C()  
]);
```

Parallel

একাধিক কাজ একই সময়ে আলাদা execution resource-এ সত্যিকার অর্থে চলছে।

JavaScript-এর সাধারণ Promise-based network I/O-কে সরাসরি "parallel threads" বলা ঠিক নয়।

মনে রাখো

`Promise.all()` সাধারণত concurrency coordinate করে; actual parallelism environment-এর ওপর নির্ভর করে।

5. Promise Combinators — Advanced View

আমরা আগের document-এ দেখেছি:

```
Promise.all()
Promise.allSettled()
Promise.race()
Promise.any()
```

এখন এগুলোকে architecture-এর দিক থেকে দেখো।

5.1 `Promise.all()`

সবগুলো successful হওয়া দরকার।

```
const [user, posts] = await Promise.all([
  getUser(),
  getPosts()
]);
```

একটি reject করলে returned Promise reject করে।

5.2 `Promise.allSettled()`

সব operation-এর final outcome দরকার।

```
const results = await Promise.allSettled([
  sendEmail(),
  sendSMS(),
```

```
    sendPush()  
  });
```

তারপর:

```
for (const result of results) {  
  if (result.status === "fulfilled") {  
    console.log("Success:", result.value);  
  } else {  
    console.error("Failed:", result.reason);  
  }  
}
```

5.3 Promise.race()

প্রথম settled Promise-এর outcome নেয়।

```
const result = await Promise.race([  
  requestA(),  
  requestB()  
]);
```

প্রথমটি success অথবা failure—দুটোর যেকোনোটি হতে পারে।

5.4 Promise.any()

প্রথম successful Promise-এর result নেয়।

```
const result = await Promise.any([  
  serverA(),  
  serverB(),  
  serverC()  
]);
```

সবগুলো reject করলে:

AggregateError

হবে।

6. Fail-Fast Architecture

Fail-fast মানে:

গুরুত্বপূর্ণ কোনো operation fail করলে পুরো operation দ্রুত fail করবে।

Example:

```
const [user, payment] = await Promise.all([
  getUser(),
  processPayment()
]);
```

যদি payment fail করে, overall operation failure হিসেবে ধরা হবে।

এটা useful যখন সব dependency success হওয়া mandatory।

7. Fail-Safe Architecture

Fail-safe মানে:

একটি non-critical operation fail করলেও পুরো application flow যেন ভেঙে না পড়ে।

Example:

```
const results = await Promise.allSettled([
  sendAnalytics(),
  sendNotification(),
  updateMainRecord()
]);
```

ধরো analytics fail করল।

তবুও main operation চালু থাকতে পারে।

8. Critical বনাম Non-Critical Tasks

Real-world application-এ প্রথমে determine করো:

```
Critical?
↓
Yes → failure propagate
No  → isolate failure
```

Example:

Critical

- payment
- database write
- authentication
- order creation

Often non-critical

- analytics
- logging
- optional recommendation
- telemetry

এটা business logic-এর ওপর নির্ভর করবে।

9. Timeout Pattern

Network request কখনো কখনো অনেকক্ষণ আটকে থাকতে পারে।

তাই timeout দরকার হতে পারে।

Concept:

```
Request
↓
wait
↓
timeout reached?
↓
cancel/fail
```

Modern JavaScript-এ timeout-এর সঙ্গে `AbortController` খুব useful।

10. AbortController

`AbortController` asynchronous operation cancellation `signal` পাঠাতে ব্যবহার করা যায়।

Basic:

```
const controller = new AbortController();  
  
const signal = controller.signal;
```

তারপর API request:

```
fetch("/api/users", {  
  signal  
});
```

Cancel:

```
controller.abort();
```

11. AbortSignal

`AbortController` থেকে পাওয়া যায়:

```
const controller = new AbortController();  
  
console.log(controller.signal);
```

`signal` operation-কে জানায়:

"এই operation abort করার request এসেছে।"

12. Fetch Cancellation

```
async function loadUsers() {  
  const controller = new AbortController();
```

```

try {
  const response = await fetch("/api/users", {
    signal: controller.signal
  });

  return await response.json();
} catch (error) {
  console.error(error);
}
}

```

Cancel করতে:

```
controller.abort();
```

13. Timeout with `AbortSignal.timeout()`

Modern environments-এ:

```

const response = await fetch("/api/users", {
  signal: AbortSignal.timeout(5000)
});

```

মানে roughly:

5 seconds-এর মধ্যে operation complete না হলে abort signal trigger হবে।

এটা manual timer + controller pattern-এর চেয়ে অনেক cleaner।

14. Manual Timeout Pattern

Conceptual implementation:

```

async function fetchWithTimeout(url, timeout) {
  const controller = new AbortController();

  const timer = setTimeout(() => {
    controller.abort();
  }, timeout);

```

```
    }, timeout);

    try {
      const response = await fetch(url, {
        signal: controller.signal
      });

      return response;
    } finally {
      clearTimeout(timer);
    }
  }
}
```

এখানে `finally` cleanup নিশ্চিত করছে।

15. Timeout ≠ Cancellation

এগুলো related কিন্তু identical নয়।

Timeout

নির্দিষ্ট সময় পার হয়ে গেছে।

Cancellation

operation বন্ধ করার request দেওয়া হয়েছে।

Timeout cancellation trigger করতে পারে।

16. Retry Pattern

Network request temporary failure করতে পারে।

যেমন:

```
Request
↓
Failed
↓
Retry
↓
```

Failed
↓
Retry
↓
Success

Basic retry:

```
async function retry(fn, attempts = 3) {  
  for (let i = 1; i <= attempts; i++) {  
    try {  
      return await fn();  
    } catch (error) {  
      if (i === attempts) {  
        throw error;  
      }  
    }  
  }  
}
```

Usage:

```
const data = await retry(() => fetchData(), 3);
```

17. Exponential Backoff

বারবার immediately retry করা ভালো না।

Instead:

```
1st retry → 1s  
2nd retry → 2s  
3rd retry → 4s  
4th retry → 8s
```

এটাকে **exponential backoff** বলা হয়।

Basic formula:

```
delay = base × 2^attempt
```

Example:

```
const delay = 1000 * 2 ** attempt;
```

18. Retry + Backoff Example

```
function wait(ms) {
  return new Promise((resolve) => {
    setTimeout(resolve, ms);
  });
}

async function retryWithBackoff(fn, attempts = 3) {
  for (let attempt = 0; attempt < attempts; attempt++) {
    try {
      return await fn();
    } catch (error) {
      if (attempt === attempts - 1) {
        throw error;
      }

      const delay = 1000 * 2 ** attempt;

      await wait(delay);
    }
  }
}
```

19. Retry সব Error-এর জন্য নয়

এটা খুব important ।

যেমন:

```
401 Unauthorized
403 Forbidden
400 Bad Request
```

এসব সাধারণত blindly retry করার মতো error নয়।

Temporary failure:

```
network issue
temporary server failure
rate limiting
```

এসব ক্ষেত্রে retry useful হতে পারে।

Production system-এ server response ও error type অনুযায়ী retry policy design করতে হয়।

20. Jitter

একই সময়ে অনেক client retry করলে সমস্যা হতে পারে।

ধরো:

```
1000 clients
  ↓
all retry after 4 sec
  ↓
server আবার overloaded
```

এটা avoid করতে random variation যোগ করা হয়।

এটাকে **jitter** বলা হয়।

Concept:

```
const jitter = Math.random() * 500;
```

তারপর:

```
delay + jitter
```

21. Concurrency Limiting

ধরো:

```
const users = 10_000;
```

তুমি যদি:

```
await Promise.all(  
  users.map((user) => processUser(user))  
);
```

করো, তাহলে একসাথে অনেক operation start হতে পারে।

এতে:

- server overload
- memory pressure
- connection exhaustion
- rate limit
- browser limitations

হতে পারে।

22. Concurrency Limit

ধরো maximum 5টি operation একসাথে চলবে।

```
5 running  
↓  
one finishes  
↓  
next starts
```

এটাই concurrency limiting।

23. Simple Worker-Pool Pattern

```
async function processWithLimit(items, workerCount, worker) {
  let index = 0;

  async function runWorker() {
    while (true) {
      const currentIndex = index++;

      if (currentIndex >= items.length) {
        return;
      }

      await worker(items[currentIndex]);
    }
  }

  const workers = Array.from(
    { length: workerCount },
    () => runWorker()
  );

  await Promise.all(workers);
}
```

Usage:

```
await processWithLimit(
  users,
  5,
  (user) => processUser(user)
);
```

এখানে সর্বোচ্চ প্রায় 5টি worker একসাথে কাজ করবে।

24. কেন Concurrency Limiting Important?

Production application-এ:

```
Unlimited concurrency
↓
```

```
too many requests
  ↓
rate limit / overload
  ↓
failure
```

Better:

```
Controlled concurrency
  ↓
stable throughput
  ↓
less pressure
  ↓
more predictable system
```

25. Async Iteration

JavaScript-এ asynchronous data stream consume করার জন্য:

```
for await...of
```

ব্যবহার করা যায়।

Basic:

```
for await (const item of asyncIterable) {
  console.log(item);
}
```

এটা asynchronous iteration।

26. Async Iterable

একটি object asynchronousভাবে values produce করতে পারে।

Example:

```
const asyncIterable = {
  async *[Symbol.asyncIterator]() {
    yield 1;
    yield 2;
    yield 3;
  }
};
```

Consume:

```
for await (const value of asyncIterable) {
  console.log(value);
}
```

Output:

```
1
2
3
```

27. Async Generators

`async function*` দিয়ে async generator তৈরি করা যায়।

```
async function* numbers() {
  yield 1;

  await wait(1000);

  yield 2;

  await wait(1000);

  yield 3;
}
```

Consume:

```
for await (const number of numbers()) {  
  console.log(number);  
}
```

28. Async Generator Mental Model

Normal generator:

```
yield → next value
```

Async generator:

```
yield  
↓  
Promise  
↓  
next async value
```

Useful for:

- paginated APIs
- streams
- large datasets
- incremental processing
- asynchronous data sources

29. Pagination with Async Generator

ধরো API page by page data দেয়।

```
async function* fetchPages() {  
  let page = 1;  
  
  while (true) {  
    const data = await fetchPage(page);  
  
    if (data.items.length === 0) {  
      return;  
    }  
  }  
}
```

```
    }  
  
    yield data.items;  
  
    page++;  
  }  
}
```

Consume:

```
for await (const items of fetchPages()) {  
  console.log(items);  
}
```

এতে পুরো dataset একসাথে memory-তে load করার দরকার নাও হতে পারে।

30. Promise.withResolvers()

Modern JavaScript-এ:

```
Promise.withResolvers()
```

একটি Promise এবং তার:

- resolve
- reject

একসাথে দেয়।

Conceptually:

```
const {  
  promise,  
  resolve,  
  reject  
} = Promise.withResolvers();
```

এখন:

```
resolve("Success");
```

করলে:

```
promise
```

fulfill হবে।

31. Traditional Pattern বনাম `withResolvers()`

Traditional:

```
let resolvePromise;  
let rejectPromise;  
  
const promise = new Promise((resolve, reject) => {  
  resolvePromise = resolve;  
  rejectPromise = reject;  
});
```

Modern:

```
const {  
  promise,  
  resolve,  
  reject  
} = Promise.withResolvers();
```

Cleaner।

32. `Promise.withResolvers()` Use Case

যখন Promise-এর settlement function পরে অন্য জায়গা থেকে ব্যবহার করতে হয়।

Example:

```
const {
  promise,
  resolve
} = Promise.withResolvers();

setTimeout(() => {
  resolve("Done");
}, 1000);

const result = await promise;

console.log(result);
```

33. `Promise.withResolvers()` কোথায় Useful?

Possible use cases:

- custom async abstractions
- event-based APIs
- adapters
- queues
- bridging callback APIs
- manually controlled Promise lifecycle

কিন্তু simple Promise-এর জন্য অযথা ব্যবহার করার দরকার নেই।

34. Top-Level `await`

ES modules-এ function-এর বাইরে `await` ব্যবহার করা যায়।

```
const response = await fetch("/api/config");

const config = await response.json();

console.log(config);
```

এটাকে বলা হয়:

Top-level await

35. Top-Level Await কেন Useful?

Module initialization-এর সময় async dependency load করতে useful হতে পারে।

Example:

```
const config = await loadConfig();  
  
export default config;
```

36. Top-Level Await-এর Caution

Module loading-এর dependency graph-এ top-level await module initialization delay করতে পারে।

তাই:

যেখানে দরকার সেখানে ব্যবহার করবে; শুধু `await` ব্যবহার করা যায় বলে everywhere ব্যবহার করবে না।

37. Microtasks ও Macrotasks

এখন Event Loop-এর advanced অংশ।

JavaScript asynchronous callbacks broadly different queues-এর মাধ্যমে schedule হয়।

Important categories:

Microtasks

Examples:

```
Promise.then()  
Promise.catch()  
Promise.finally()  
queueMicrotask()
```

Tasks / Macrotasks

Common examples:

```
setTimeout()  
setInterval()
```

Browser environment-এ আরও task sources আছে।

38. Microtasks সাধারণত আগে Run হয়

Example:

```
console.log("A");  
  
setTimeout(() => {  
  console.log("B");  
}, 0);  
  
Promise.resolve().then(() => {  
  console.log("C");  
});  
  
console.log("D");
```

Expected:

```
A  
D  
C  
B
```

কারণ:

```
Synchronous code  
  ↓  
Microtasks  
  ↓  
Next task
```

39. Event Loop Mental Model

```
Call Stack
  ↓
Run synchronous code
  ↓
Microtask checkpoint
  ↓
Task
  ↓
Microtask checkpoint
  ↓
Next task
```

এটা conceptual model।

Browser-এর rendering এবং অন্যান্য event-loop details আরও nuanced হতে পারে।

40. queueMicrotask()

Manually microtask schedule করা যায়:

```
queueMicrotask(() => {
  console.log("Microtask");
});
```

Example:

```
console.log("A");

queueMicrotask(() => {
  console.log("B");
});

console.log("C");
```

Output:

A
C
B

41. Microtask Starvation

যদি continuously microtask schedule করা হয়:

```
function loop() {  
  queueMicrotask(loop);  
}  
  
loop();
```

তাহলে task/rendering opportunity severely delay হতে পারে।

এটাকে **microtask starvation** বলা যায়।

তাই asynchronous scheduling-এর mechanism বুঝে ব্যবহার করতে হবে।

42. Async Error Propagation

ধরো:

```
async function a() {  
  throw new Error("Failed");  
}  
  
async function b() {  
  await a();  
}  
  
async function c() {  
  await b();  
}
```

যদি `c()`-তে catch না থাকে, rejection আরও উপরে propagate করবে।

```
a()
↓
reject
↓
b()
↓
reject
↓
c()
↓
caller
```

43. Error Boundary Pattern

Application layer-এ সাধারণত lower-level function error handle না করে caller-এর কাছে propagate করতে পারে।

```
async function repository() {
  return fetchData();
}

async function service() {
  return repository();
}

async function controller() {
  try {
    return await service();
  } catch (error) {
    console.error(error);

    throw error;
  }
}
```

এতে responsibility layer অনুযায়ী ভাগ করা যায়।

44. Fire-and-Forget

কখনো এমন operation আছে যেটার result caller-এর দরকার নেই।

Example:

```
trackAnalytics(event);
```

যদি intentionally background operation হয়, তাহলে error policy define করো।

```
void trackAnalytics(event).catch((error) => {  
  console.error("Analytics failed:", error);  
});
```

কিন্তু

Critical operation কখনো fire-and-forget করা উচিত নয়।

```
void processPayment();
```

এটা dangerous হতে পারে।

45. Async Resource Cleanup

Async operation-এর সঙ্গে resource cleanup গুরুত্বপূর্ণ।

যেমন:

- timer
- event listener
- network connection
- file handle
- subscription
- stream
- lock

Pattern:

```
try {  
  await useResource();  
} finally {  
  await cleanupResource();  
}
```

46. Why `finally` Matters

```
async function process() {  
  const resource = await acquire();  
  
  try {  
    await work(resource);  
  } finally {  
    await release(resource);  
  }  
}
```

Success:

```
work  
↓  
release
```

Failure:

```
work  
↓  
error  
↓  
release
```

তাই cleanup reliable হয়।

47. Abort + Cleanup

```
async function loadData(signal) {  
  const response = await fetch("/api/data", {  
    signal  
  });  
  
  return response.json();  
}
```

Caller:

```
const controller = new AbortController();

try {
  const data = await loadData(controller.signal);
} finally {
  controller.abort();
}
```

Actual architecture অনুযায়ী abort timing আলাদা হতে পারে; এখানে pattern বোঝানো হয়েছে।

48. Promise-Based Event Pattern

কিছু callback/event-based API-কে Promise-based interface-এ convert করা যায়।

Example:

```
function waitForEvent(emitter, eventName) {
  return new Promise((resolve) => {
    emitter.once(eventName, resolve);
  });
}
```

তারপর:

```
const data = await waitForEvent(emitter, "data");
```

এতে callback/event style API:

event → callback

থেকে:

event → Promise → await

এ convert করা যায়।

49. Callback API থেকে Promise Adapter

ধরো old API:

```
readFile(path, (error, data) => {  
  if (error) {  
    // error  
  }  
  
  // data  
});
```

এটাকে Promise wrapper দিয়ে modern interface দেওয়া যায়।

```
function readFileAsync(path) {  
  return new Promise((resolve, reject) => {  
    readFile(path, (error, data) => {  
      if (error) {  
        reject(error);  
        return;  
      }  
  
      resolve(data);  
    });  
  });  
}
```

তারপর:

```
const data = await readFileAsync(path);
```

50. Async State Machine

Complex async operation-এর state explicitly represent করা useful।

Example:

```
const state = {  
  status: "idle",
```

```
data: null,  
error: null  
};
```

Possible states:

```
idle  
loading  
success  
error
```

Flow:

```
idle  
↓  
loading  
↓  
success
```

অথবা:

```
idle  
↓  
loading  
↓  
error
```

51. কেন Async State Machine Useful?

UI application-এ:

```
loading  
success  
error  
empty  
retrying
```

এসব state handle করতে হয়।

Explicit state model race condition এবং impossible state কমাতে সাহায্য করতে পারে।

52. Async Race Condition

ধরো user দ্রুত search করছে:

```
Request A → "react"  
Request B → "react hooks"  
Request C → "react hooks tutorial"
```

কিন্তু response order হতে পারে:

```
C → finish  
A → finish  
B → finish
```

যদি A-এর response পরে এসে UI update করে, তাহলে stale data দেখাতে পারে।

53. Race Condition Prevent করার একটি উপায়

Previous request cancel করা।

```
let controller;  
  
async function search(query) {  
  controller?.abort();  
  
  controller = new AbortController();  
  
  const response = await fetch(  
    `/api/search?q=${encodeURIComponent(query)}`,  
    {  
      signal: controller.signal  
    }  
  );  
  
  return response.json();  
}
```

নতুন request শুরু হলে আগেরটা abort করার চেষ্টা করা হচ্ছে।

54. Another Race-Safety Pattern

Request ID ব্যবহার করা যায়।

```
let requestId = 0;

async function search(query) {
  const id = ++requestId;

  const data = await fetchSearch(query);

  if (id !== requestId) {
    return;
  }

  renderResults(data);
}
```

এখানে latest request ছাড়া পুরোনো response UI update করতে পারবে না।

55. Async Queue Concept

ধরো কাজগুলো:

```
Task A
Task B
Task C
Task D
```

Queue:

```
A → B → C → D
```

Worker:

```
Worker
  ↓
take task
  ↓
process
```

↓
take next

এটা concurrency control-এর foundation ।

56. Backpressure

যখন producer খুব দ্রুত data তৈরি করছে কিন্তু consumer ধীরে process করছে, তখন সমস্যা হয় ।

Producer



↓

Consumer

Queue ক্রমাগত বড় হতে পারে ।

এটাকে **backpressure problem** বলা হয় ।

Streams এবং async iterators-এ backpressure handling খুব important advanced concept ।

57. Async Iteration বনাম `Promise.all()`

`Promise.all()`

সব operation একসাথে শুরু করতে ভালো ।

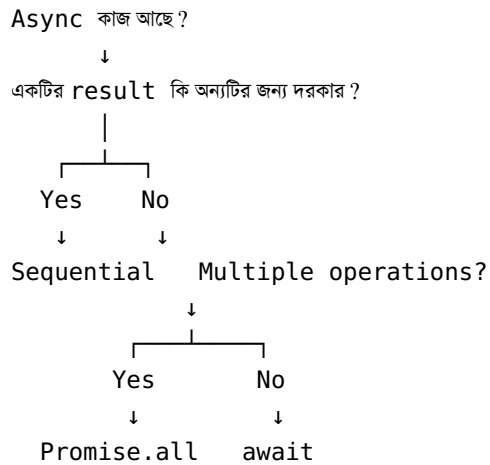
```
const results = await Promise.all(
  items.map(processItem)
);
```

Async iteration

একটি async source থেকে ধাপে ধাপে data consume করতে ভালো ।

```
for await (const item of source) {
  await processItem(item);
}
```

58. Async Architecture — Practical Decision Tree



তারপর:

সব success দরকার ?
→ `Promise.all()`

সব outcome দরকার ?
→ `Promise.allSettled()`

First settled দরকার ?
→ `Promise.race()`

First successful দরকার ?
→ `Promise.any()`

59. Production Async Checklist

একটি async function লিখার আগে ভাবো:

1. Dependency আছে?

A → B

তাহলে sequential।

2. Independent?

A
B
C

তাহলে concurrency consider করো।

3. Failure critical?

হলে fail-fast।

4. Failure optional?

হলে fail-safe pattern consider করো।

5. Timeout দরকার?

Long-running external operation হলে consider করো।

6. Cancellation দরকার?

User navigation, search, component lifecycle ইত্যাদিতে useful হতে পারে।

7. Retry দরকার?

Transient failure হলে retry policy consider করো।

8. Rate limit আছে?

Concurrency limit দরকার হতে পারে।

9. Cleanup আছে?

`finally` consider করো।

60. Practice — Easy

Question 1

কোনটি sequential?

```
const a = await getA();  
const b = await getB();
```

নাকি:

```
const [a, b] = await Promise.all([  
  getA(),  
  getB()  
]);
```

Question 2

`Promise.any()` কখন reject করে?

Question 3

`Promise.race()` কি প্রথম successful Promise-এর জন্য wait করে?

61. Practice — Medium

Question 4

এই code-এর সমস্যা identify করো:

```
items.forEach(async (item) => {  
  await processItem(item);  
});  
  
console.log("Done");
```

Better version লিখো।

Question 5

নিচের operations-এর dependency বের করো:

```
getUser()  
getOrders(userId)  
getNotifications(userId)  
getRecommendations()
```

সবচেয়ে efficient structure লিখো।

62. Practice — Advanced

Question 6

তোমার application-এ 10,000 users আছে।

```
await Promise.all(  
  users.map((user) => sendEmail(user))  
);
```

এখানে কী কী সমস্যা হতে পারে?

কীভাবে maximum 10টি email operation একসাথে চলবে এমন system design করবে?

Question 7

একটি search input আছে।

User দ্রুত টাইপ করছে:

```
r  
re  
rea  
reac  
react
```

প্রতিটি input-এর জন্য API call হচ্ছে।

পুরোনো response যেন নতুন result overwrite না করে—দুটি solution লিখো।

63. Practice — Output Prediction

```
console.log("A");

setTimeout(() => {
  console.log("B");
}, 0);

Promise.resolve().then(() => {
  console.log("C");
});

queueMicrotask(() => {
  console.log("D");
});

console.log("E");
```

Output predict করো।

64. Practice — Real-World Challenge

একটি dashboard load করতে হবে।

Requirements:

1. User information mandatory
2. User পাওয়ার পর Orders load হবে
3. Notifications user ID দিয়ে load হবে
4. Recommendations independent
5. Analytics optional
6. Dashboard 5 seconds-এর মধ্যে response না পেলে timeout

তুমি এমন architecture design করো যেখানে:

- dependency maintain হবে
- independent operations concurrent হবে
- optional analytics dashboard fail করবে না
- timeout থাকবে
- proper cleanup থাকবে

65. Interview Questions

Core

1. Floating Promise কী?
2. Unhandled rejection কী?
3. Sequential vs concurrent কী?
4. Concurrency vs parallelism কী?
5. `Promise.all()` কি operation cancel করে?
6. Fail-fast কী?
7. Fail-safe কী?

Advanced

1. `AbortController` কী?
2. Timeout আর cancellation-এর পার্থক্য কী?
3. Exponential backoff কী?
4. Jitter কেন ব্যবহার করা হয়?
5. Concurrency limiting কেন দরকার?
6. Async iterator কী?
7. Async generator কী?
8. `for await...of` কী?
9. `Promise.withResolvers()` কী?
10. Top-level await কী?
11. Microtask কী?
12. Task/macrotask কী?
13. Microtask starvation কী?
14. Async race condition কী?
15. Backpressure কী?
16. Async resource cleanup কেন গুরুত্বপূর্ণ?
17. Promise-based adapter কী?

66. Quick Revision

Async Function

```
async function test() {  
  return 10;  
}
```

→ `Promise<number>`

Await

```
const result = await promise;
```

→ Promise-এর fulfilled value পাওয়া যায়।

Sequential

```
await A();  
await B();  
await C();
```

→ dependency/order থাকলে।

Concurrent

```
await Promise.all([  
  A(),  
  B(),  
  C()  
]);
```

→ independent operations-এর জন্য।

Promise Combinators

```
all  
→ সব success দরকার  
  
allSettled  
→ সব outcome দরকার  
  
race  
→ first settled  
  
any  
→ first fulfilled
```

Cancellation

```
const controller = new AbortController();

fetch(url, {
  signal: controller.signal
});

controller.abort();
```

Timeout

```
fetch(url, {
  signal: AbortSignal.timeout(5000)
});
```

Retry

```
failure
↓
wait
↓
retry
↓
wait longer
↓
retry
```

Async Iteration

```
for await (const item of source) {
  console.log(item);
}
```

Async Generator

```
async function* generate() {  
  yield 1;  
  yield 2;  
}
```

Promise.withResolvers()

```
const {  
  promise,  
  resolve,  
  reject  
} = Promise.withResolvers();
```

Microtask

```
Promise.resolve().then(...)  
queueMicrotask(...)
```

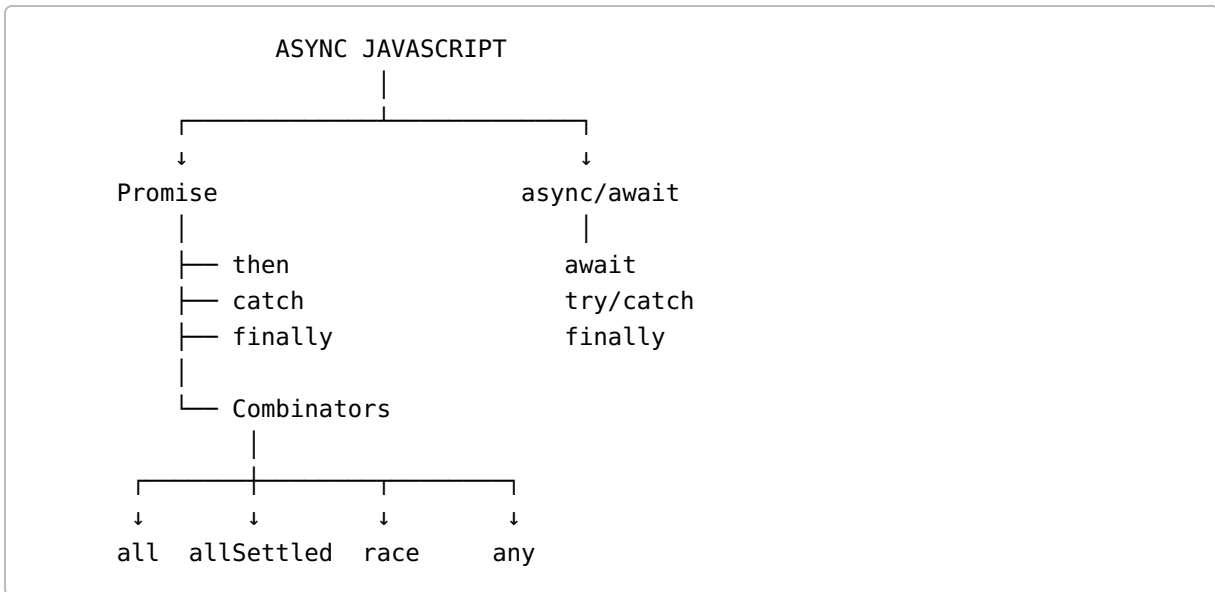
সাধারণভাবে next task-এর আগে process হয়।

Cleanup

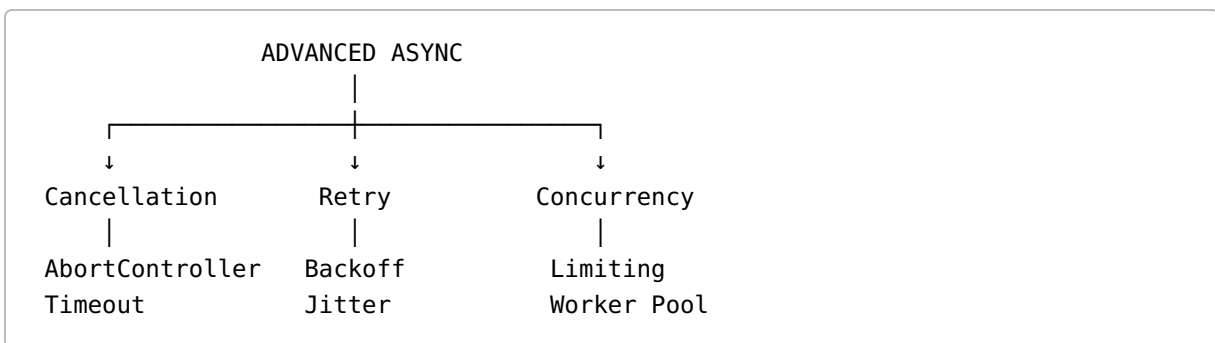
```
try {  
  await work();  
} finally {  
  await cleanup();  
}
```

67. Async JavaScript Master Mental Model

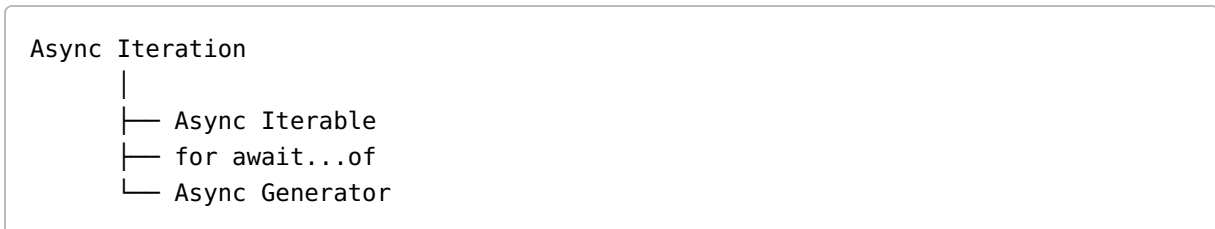
পুরো Async JavaScript-কে এইভাবে চিন্তা করো:



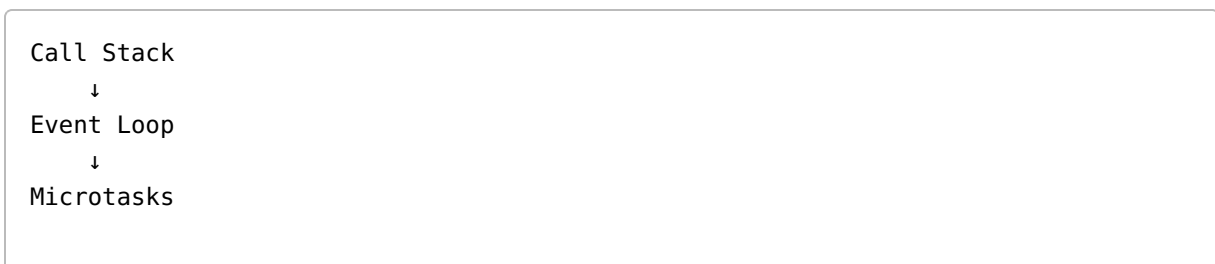
Advanced layer:



আর:



আর execution model:



68. Final Mastery Checklist

এই পুরো Async JavaScript section শেষ করার পর তুমি যেন explain করতে পারো:

- ☐ Callback
- ☐ Callback hell
- ☐ Inversion of control
- ☐ Promise
- ☐ Promise states
- ☐ Promise chaining
- ☐ Error propagation
- ☐ `async`
- ☐ `await`
- ☐ `try/catch/finally`
- ☐ `Promise.all`
- ☐ `Promise.allSettled`
- ☐ `Promise.race`
- ☐ `Promise.any`
- ☐ `AggregateError`
- ☐ Floating Promise
- ☐ Unhandled rejection
- ☐ Sequential execution
- ☐ Concurrent execution
- ☐ Parallelism
- ☐ `for...of` + `await`
- ☐ `map()` + `Promise.all()`
- ☐ `forEach(async...)` problem
- ☐ Fail-fast
- ☐ Fail-safe
- ☐ Timeout
- ☐ `AbortController`
- ☐ `AbortSignal`
- ☐ Cancellation
- ☐ Retry
- ☐ Exponential backoff
- ☐ Jitter
- ☐ Concurrency limiting
- ☐ Async iteration
- ☐ Async generators
- ☐ `for await...of`
- ☐ `Promise.withResolvers()`

- [] Top-level await
 - [] Microtasks
 - [] Tasks/macrotasks
 - [] Microtask starvation
 - [] Async error propagation
 - [] Fire-and-forget
 - [] Resource cleanup
 - [] Promise-based event patterns
 - [] Async state machines
 - [] Race conditions
 - [] Backpressure
-

69. The Most Important Rules

শেষে শুধু এই rules-গুলো মাথায় রাখলেই Async JS-এর অনেকটা control চলে আসবে:

Rule 1

Dependency থাকলে **sequential**.

```
const user = await getUser();
const orders = await getOrders(user.id);
```

Rule 2

Independent হলে **concurrency consider** করো.

```
const [a, b, c] = await Promise.all([
  getA(),
  getB(),
  getC()
]);
```

Rule 3

`forEach(async...)` দিয়ে **async workflow control** করো না।

Sequential:

```
for (const item of items) {  
  await process(item);  
}
```

Concurrent:

```
await Promise.all(  
  items.map(process)  
);
```

Rule 4

Critical operation-এর **error silently ignore** করো না।

Rule 5

Long-running external operation-এ **timeout/cancellation consider** করো।

Rule 6

Retry মানেই সব **error retry** করা নয়।

Rule 7

Unlimited concurrency production system-এ **dangerous** হতে পারে।

Rule 8

Resource cleanup-এর জন্য `finally` **powerful**।

Rule 9

`Promise.all()` **rejection** অন্য **already-started operation automatically cancel** করে না।

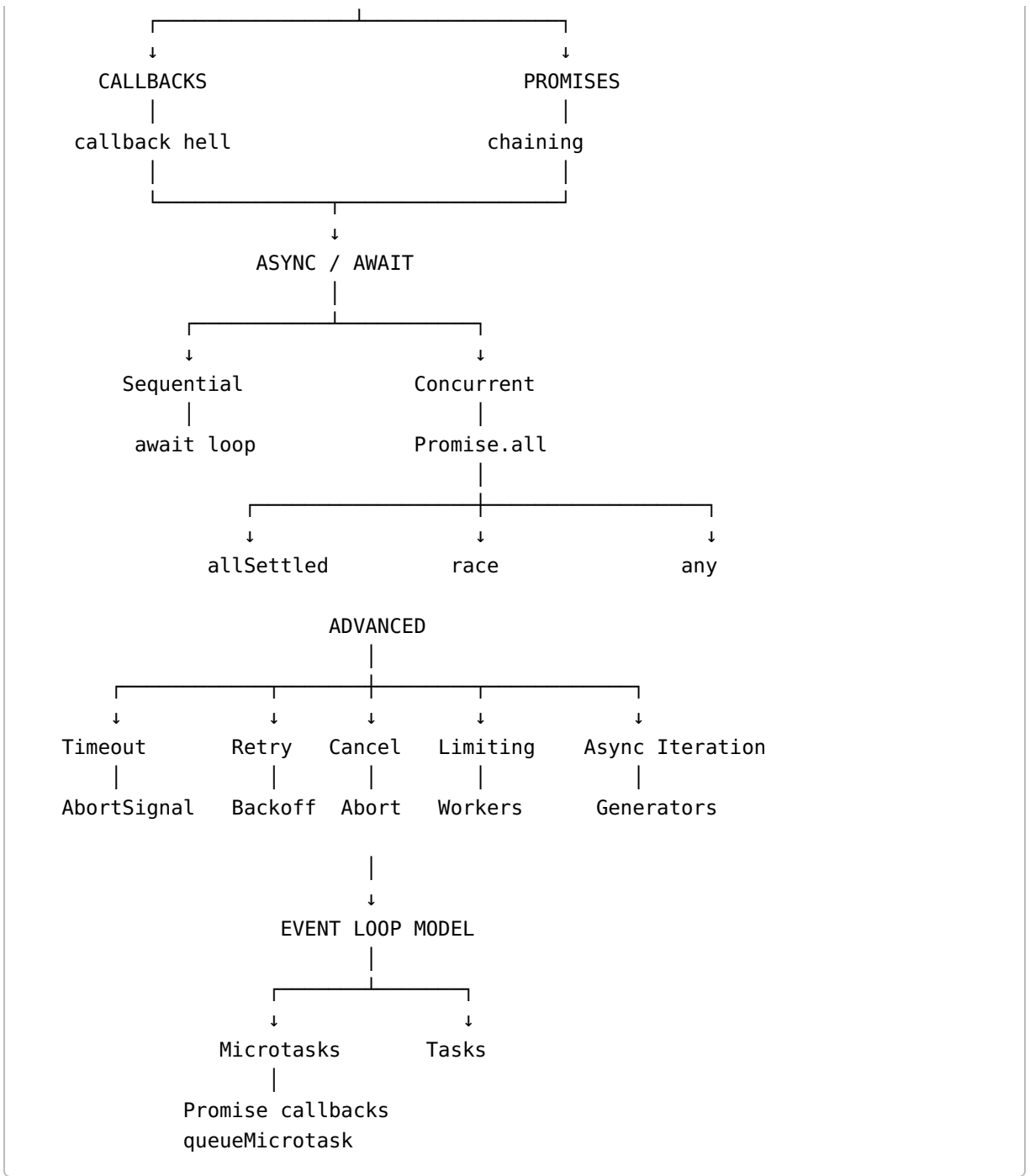
Rule 10

`await` পুরো **JavaScript thread block** করে না।

Final Async JavaScript Picture

ASYNC JAVASCRIPT

|



Async JavaScript section এখানেই complete.